12.10.2026

Einschub:
Eine echte Aufgabe



Aufgabe

Winter 2024/2025

Anlage zur 4. Aufgabe

Beispiele für verschiedene Schleifenarten:

```
while(number < 5)
  {
     Console.WriteLine(number);
     number = number + 1
  }

for(int i = 0; i < number; i++)
  {
     Console.WriteLine(number);
  }

do
  {
    Console.WriteLine(number);
    number = number + 1;
  } while(number < 5);
```

Beispiel Wertzuweisung:

```
i = i+3 kann als i += 3 in Kurzform geschrieben werden.
```

Beispiel für eine Auswahlanweisung:

```
int number = 20;
if (number < 18)
{
  Console.WriteLine("ok");
}
else
{
  Console.WriteLine("not ok");
}
```

Kommentare

```
//Kommentar einzeilig
/* Kommentar mehrzeilig */
```

Beispiel für Konsolenausgabe

```
Console.WriteLine("ok");    //Ausgabe von Text
Console.WriteLine(Zahl);    //Ausgabe einer Variablen
```

Aufgabe

Winter 2024/2025

4. Aufgabe (25 Punkte)

Sie wirken bei der Entwicklung von Skripten zur Systemverwaltung mit und sollen folgende, aus prüfungstechnischen Gründen stark vereinfachte Aufgabe erledigen.

a) Ein Monitoring-Programm soll um die Methode „`Mittelwerte`" erweitert werden, welche von jeweils drei aufeinanderfolgenden Werten im Array PerfCPU den Mittelwert berechnet. Gestartet wird mit den ersten drei Werten „40, 42, 47" in PerfCPU. Der Mittelwert beträgt „43" und ersetzt die „42", die Werte „40" und „47" werden mit „0" überschrieben.

Aus den ursprünglichen Werten 40, 42, 47" wird somit „0, 43, 0".

Danach wird mit den jeweils nächsten drei Werten bis zum Ende des Arrays entsprechend verfahren.

Array PerfCPU: Inhalt vor Durchlauf der Methode „`Mittelwerte`"

Index	0	1	2	3	4	5	6	7	8
Wert	**40**	**42**	**47**	**52**	**50**	**60**	**76**	**77**	**81**

Array PerfCPU: gewünschter Inhalt nach Durchlauf der Methode „`Mittelwerte`"

Index	0	1	2	3	4	5	6	7	8
Wert	**0**	**43**	**0**	**0**	**54**	**0**	**0**	**78**	**0**

Aufgabe

Winter 2024/2025

4. Aufgabe (25 Punkte)

Sie wirken bei der Entwicklung von Skripten zur Systemverwaltung mit und sollen folgende, aus prüfungstechnischen Gründen stark vereinfachte Aufgabe erledigen.

a) Ein Monitoring-Programm soll um die Methode „`Mittelwerte`" erweitert werden, welche von jeweils drei aufeinanderfolgenden Werten im Array PerfCPU den Mittelwert berechnet. Gestartet wird mit den ersten drei Werten „40, 42, 47" in PerfCPU. Der Mittelwert beträgt „43" und ersetzt die „42", die Werte „40" und „47" werden mit „0" überschrieben.

Aus den ursprünglichen Werten 40, 42, 47" wird somit „0, 43, 0".

Danach wird mit den jeweils nächsten drei Werten bis zum Ende des Arrays entsprechend verfahren.

Array PerfCPU: Inhalt vor Durchlauf der Methode „`Mittelwerte`"

Index	0	1	2	3	4	5	6	7	8
Wert	**40**	**42**	**47**	**52**	**50**	**60**	**76**	**77**	**81**

Array PerfCPU: gewünschter Inhalt nach Durchlauf der Methode „`Mittelwerte`"

Index	0	1	2	3	4	5	6	7	8
Wert	**0**	**43**	**0**	**0**	**54**	**0**	**0**	**78**	**0**

Aufgabe

Winter 2024/2025

aa) Folgender Quellcode liegt vor:

```
Zeile1   //Test-Array anlegen und mit Test-Daten füllen
Zeile2   int[] PerfCPU = new int[] {40,42,47,52,50,60,76,77,81};
Zeile3   //Variable deklarieren
Zeile4   int temp = 0;
Zeile5   //Schleife zum Berechnen und Speichern der Mittelwerte
Zeile6   for (int i = 1; i < 6; i += 3)
Zeile7     {
Zeile8       temp = ((PerfCPU [i - 1]) + (PerfCPU [i]) + (PerfCPU [i + 1])) / 3;
Zeile9       PerfCPU [i] = 0; PerfCPU [i-1] = temp; PerfCPU[i + 1] = 0;
Zeile10    }
```

Stellen Sie fest, ob der Quellcode entsprechend der Beschreibung die gewünschten Ergebnisse liefert. Analysieren Sie dazu schrittweise den Quellcode und tragen Sie die ermittelten Werte in die vorgesehenen Felder ein. 9 Punkte

Hinweis: In der perforierten Anlage auf Seite 9 finden Sie Erläuterungen zu der verwendeten Syntax.

Array PerfCPU: Inhalt nach Durchlauf der Methode „`Mittelwerte`"

Array-Index	0	1	2	3	4	5	6	7	8
Array-**Wert**									

Aufgabe

Winter 2024/2025

ab) Der Programmcode liefert offensichtlich nicht die gewünschten Werte. Dafür verantwortlich sind **zwei** fehlerhafte Anweisungen im Programmcode.

Geben Sie die **zwei** Anweisungen in korrekter Form an. 10 Punkte

Programmzeile	**Anweisung**

Aufgabe

Winter 2024/2025

b) Die Daten des Arrays sollen zusammen mit weiteren Daten in einer CSV-Datei gespeichert werden.

ba) Erläutern Sie zwei Argumente, die für die Verwendung von CSV-Dateien sprechen. 4 Punkte

Aufgabe

Winter 2024/2025

bb) Erläutern Sie ein Argument, das gegen die Verwendung von CSV-Dateien spricht. 2 Punkte

Objektorientierung

Warum objektorientiert?

- Die reale Welt besteht aus Objekten (Auto, Konto, Kunde).
- OOP fasst Daten und Verhalten zusammen, statt sie zu trennen.
- Das Programm ist näher an der Wirklichkeit und wird übersichtlicher.

Klasse und Objekt

- Klasse: der Bauplan beschreibt Aussehen und Fähigkeiten.
- Objekt: eine konkrete Ausprägung der Klasse (Instanz).

Eine Klasse, viele Objekte (wie ein Bauplan für viele Häuser).



Attribute

- Die Eigenschaften eines Objekts: sein Zustand.
- Beispiel für die Klasse „Konto“: `inhaber`, `kontostand`

- Jedes Objekt hat eigene Werte für seine Attribute.

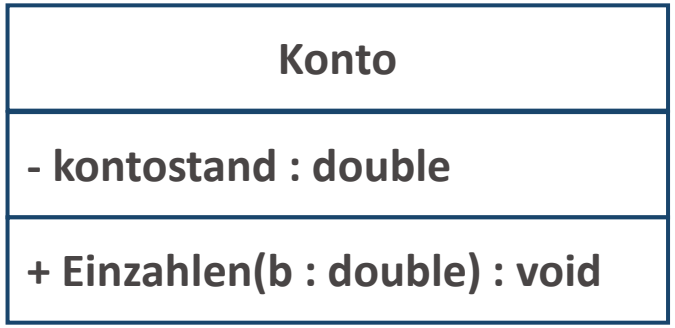
Methoden

- Das Verhalten eines Objekts: was es kann.
- Beispiel Konto: `Einzahlen()`, `Abheben()`

- Methoden arbeiten mit den Attributen des Objekts.

Beispiel: Klasse

```
class Konto
{
  private double kontostand;
  public void Einzahlen(double b)
  {
    kontostand = kontostand + b;
  }
}
```



Vorteile der OOP

- **Struktur:** Daten und Verhalten gehören zusammen.
- **Wiederverwendung:** Klassen lassen sich erneut nutzen.
- **Wartbarkeit:** Änderungen bleiben lokal in der Klasse.

Die **Nähe zur Realwelt** erleichtert das Modellieren.

Kernkonzepte

- Kapselung
- Abstraktion
- Vererbung
- Polymorphismus

Kapselung bezeichnet die Zusammenfassung von Daten (Variablen) und Verhalten (Methoden) in einer einzigen Einheit, typischerweise einer Klasse.

Die Kapselung verhindert den direkten Zugriff auf Daten.

Daher sind Änderungen nur über bestimmte Methoden möglich. Dies ist entscheidend für die Datenintegrität und -sicherheit.

Kernkonzepte

- Kapselung
- Abstraktion
- Vererbung
- Polymorphismus

Abstraktion bedeutet, unnötige Details zu verbergen und nur die wesentlichen Merkmale eines Objekts darzustellen.

Dies erleichtert das Schreiben und Verwenden von Code und reduziert zudem die Komplexität der Programmierung.

Kernkonzepte

- Kapselung
- Abstraktion
- Vererbung
- Polymorphismus

Vererbung ermöglicht es neuen Klassen, Eigenschaften und Methoden von bestehenden Klassen zu übernehmen.

Zudem fördert sie die Wiederverwendung von Code und schafft eine natürliche Hierarchie.

Kernkonzepte

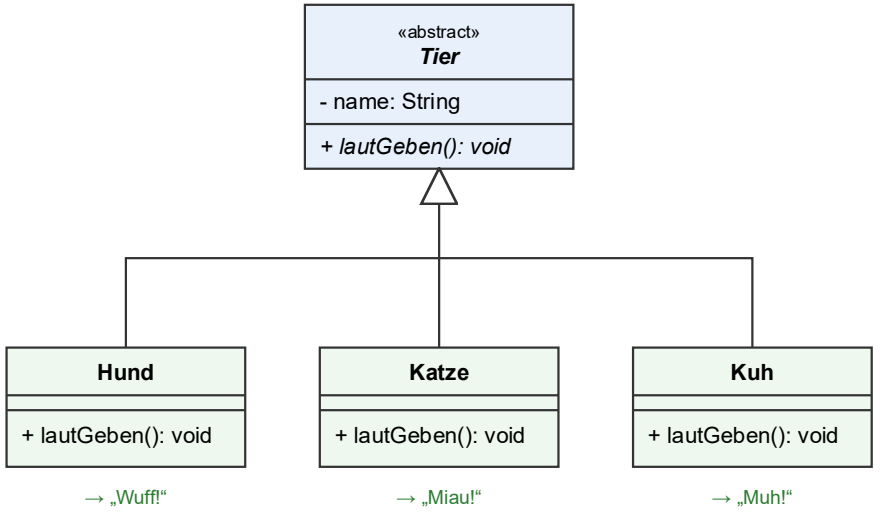
- Kapselung
- Abstraktion
- Vererbung
- Polymorphismus

Polymorphismus bedeutet „viele Formen“ und ermöglicht es, dass eine einzige Schnittstelle verschiedene zugrunde liegende Datentypen repräsentiert.

Polymorphismus bedeutet „Vielgestaltigkeit“: Ein und derselbe Befehl führt bei verschiedenen Objekten zu unterschiedlichem Verhalten.

Polymorphie

- Stellen Sie sich eine Oberklasse „Tier“ mit der Methode lautGeben() vor.
- Die Unterklassen „Hund“ und „Katze“ überschreiben diese Methode: Der Hund bellt, die Katze miaut.
- Wenn Sie nun eine Liste von Tieren durchgehen und bei jedem lautGeben() aufrufen, müssen Sie nicht wissen, welches Tier es genau ist.
- Jedes Objekt reagiert auf seine eigene Art.



Gleicher Aufruf lautGeben() – unterschiedliches Verhalten je nach Objekt

Finally…
Good to know

Hoch- und Maschinensprache

- Maschinensprache besteht nur aus 0 und 1, also dem Einzigen, was der Prozessor direkt versteht.
- Eine Hochsprache hingegen ist für Menschen lesbar (C#, Java, Python) und erlaubt uns, große Programme zu erstellen.

Dazwischen ist immer eine Übersetzung nötig.

Compiler und Interpreter

- Ein Compiler übersetzt das ganze Programm vorab → schnelle Ausführung.
- Ein Interpreter übersetzt Zeile für Zeile zur Laufzeit → flexibler, aber langsamer.

Merke: C# ist eine Mischform, denn der Compiler erzeugt aus unserem Programm einen Zwischencode (die s. g. Intermediate Language, IL), der zur Laufzeit ausgeführt wird.

Schreibtischtest

- Ein Programm per Hand durchspielen.
- Man verfolgt, welche Werte die Variablen Schritt für Schritt annehmen.
- Ziel: die Logik prüfen und Fehler finden.



Wozu ein Schreibtischtest?

- Fehler finden, bevor das Programm überhaupt läuft.
- Den Ablauf verstehen, besonders bei Schleifen.
- Das Endergebnis vorhersagen und überprüfen.



Themenblock 12
Systemmodellierung mit UML

Was ist UML?

UML (Unified Modeling Language) ist eine standardisierte grafische Sprache zur Modellierung, Visualisierung und Planung von Softwarestrukturen und -abläufen vor der eigentlichen Umsetzung.

Ursprung

- Der UML (Unified Modeling Language) Standard wird von der Object Management Group (OMG) entwickelt, gepflegt und herausgegeben (siehe dazu auch omg.org/spec/UML).
- UML liegt aktuell in der Version 2.5.1 (veröffentlicht in 12/2017) vor und wurde zudem als ISO/IEC 19505 Norm (die allerdings auf UML 2.4.1 aufbaut) standardisiert.



UML besteht aus 14 Diagrammtypen, die man in Struktur- und Verhaltensdiagramme gliedert.

Strukturdiagramme

- Klassendiagramm
- Objektdiagramm
- Komponentendiagramm
- Kompositionsstrukturdiagramm
- Paketdiagramm
- Verteilungsdiagramm
- Profildiagramm

Verhaltensdiagramme

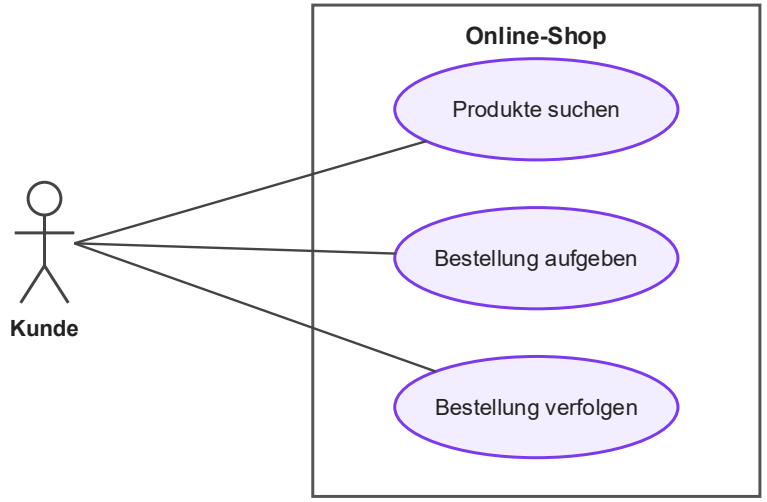
- Anwendungsfalldiagramm (Use-Case-Diagramm)
- Aktivitätsdiagramm
- Zustandsdiagramm
- Sequenzdiagramm
- Kommunikationsdiagramm
- Zeitverlaufsdiagramm
- Interaktionsübersichtsdiagramm

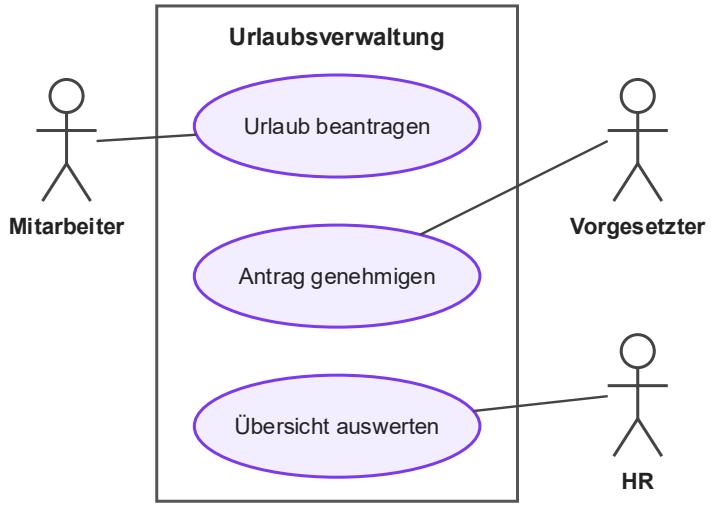
Verhaltensdiagramme

Use-Case-Diagramm

Einleitung

- Ein Use-Case-Diagramm zeigt, was ein System für wen leistet, aber nicht wie es das tut.
- Es ist Blick von außen (Nutzersicht statt Technik) und dient als gemeinsame Grundlage für Fachbereich, Entwicklung und Test.
- Typischer Einsatz: frühe Anforderungsanalyse und Abgrenzung des Scopes.





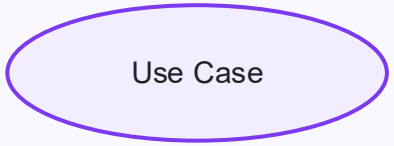
Die Grundelemente im Überblick



Akteur

wer interagiert

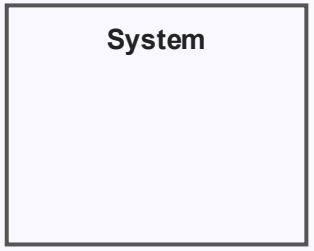
Akteure werden als „Strich-männchen" dargestellt, welche sowohl Personen wie Kunden oder Administratoren als auch ein System darstellen können.



Use Case

was das System leistet

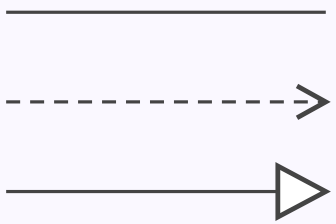
Anwendungsfälle werden in Ellipsen dargestellt. Sie müssen (z. B. in einem Kommentar oder eigenen Datei) beschrieben werden.



Systemgrenze

was zum System gehört

Die Systemgrenze bzw. der Systemkontext wird durch Systemgrenzen in Form von Rechtecken gekennzeichnet.



Beziehungen

wie alles zusammenhängt

Beziehungen schaffen Verbindungen zwischen den Elementen und zeigen, wie alles zusammenhängt.

Akteure

- Symbol: Strichmännchen (alternativ Rechteck mit «actor»)
- Stellen eine Rolle dar, keine konkrete Person („Mitarbeiter“, nicht „Frau Müller“)
- Stehen immer außerhalb des Systems
- Können auch externe Systeme sein (z. B. Zahlungsdienst, Kalender-System)



Use Cases

- Symbol: Ellipse mit Namen
- Beschreibt eine Funktion mit erkennbarem Nutzen für einen Akteur
- Benennung: Substantiv + Verb, z. B. „Urlaub beantragen“
- Nicht zu klein bzw. groß: weder „Button klicken“ noch „System benutzen“ ist ein Use Case



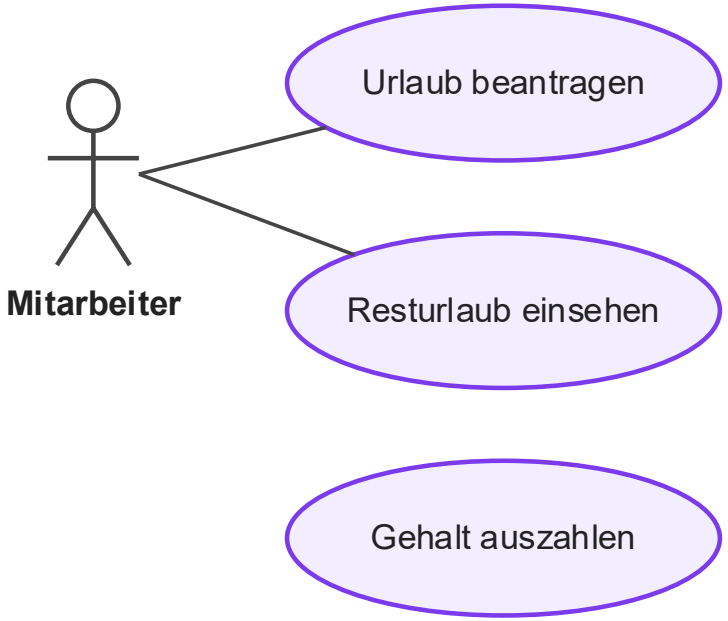
Systemgrenze

- Symbol: Rechteck mit dem Systemnamen oben
- Use Cases liegen innerhalb, Akteure liegen außerhalb
- Macht den Scope sichtbar: Was gehört zu unserem System, was nicht?
- Hilft bei der Abstimmung von Zuständigkeiten im Projekt

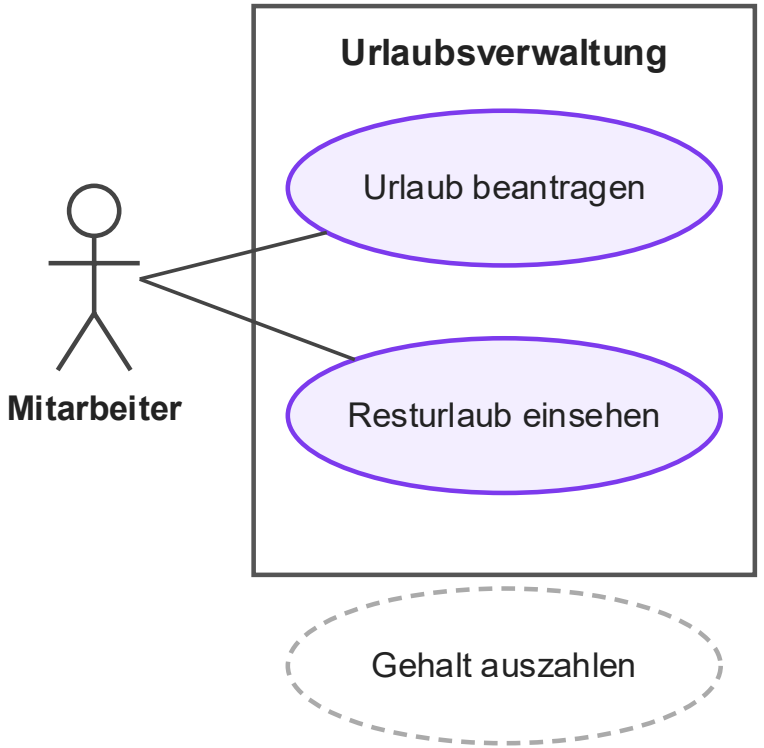


Systemgrenze

Ohne Systemgrenze: Was gehört dazu?



Mit Systemgrenze: Scope ist klar



außerhalb = nicht Teil des Systems

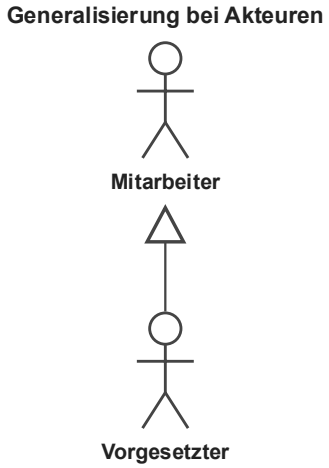
Assoziation

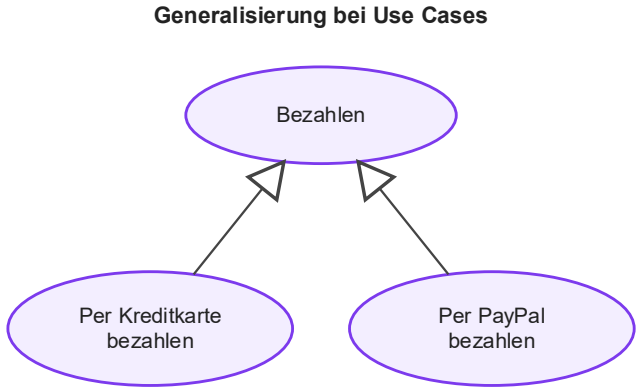
- **Assoziation:** einfache Linie zwischen Akteur und Use Case („ist beteiligt an“)



Generalisierung

- Generalisierung (Akteure): Pfeil mit leerer Dreiecksspitze
- „Vorgesetzter“ erbt von „Mitarbeiter“ – kann alles, was ein Mitarbeiter kann, plus mehr
- Generalisierung (Use Cases): ebenfalls Pfeil mit leerer Dreiecksspitze
- „Bezahlen“ → „Per Kreditkarte bezahlen“ / „Per PayPal bezahlen“



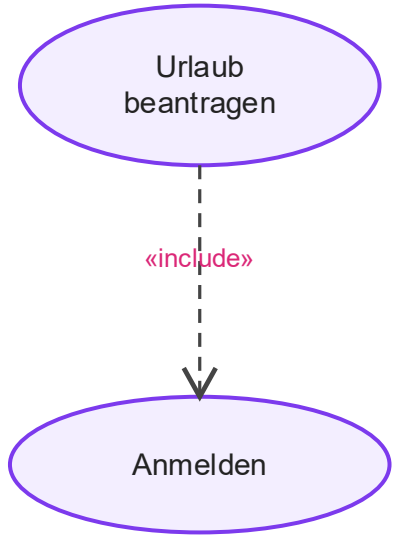


«include»-Beziehung

- Symbol: gestrichelter Pfeil
- Richtung: vom Basis- zum eingebundenen Use Case

«include» wird immer ausgeführt.

«include» – wird immer ausgeführt



Pfeil: Basis → eingebundener Use Case

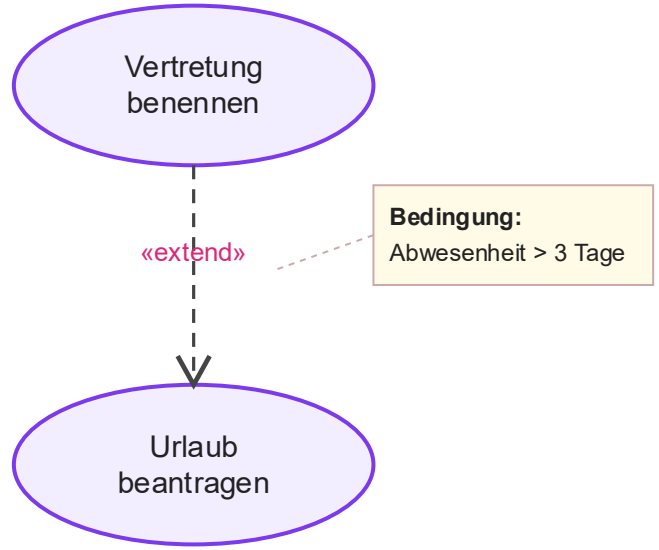
«extend»-Beziehung

- Symbol: gestrichelter Pfeil
- Richtung: vom erweiternden zum Basis-Use-Case

«extend» wird optional bzw. mit Bedingung ausgeführt.

- Extension Point: Stelle und Bedingung der Erweiterung (als Notiz oder im Use Case)

«extend» – optional / unter Bedingung



Pfeil: Erweiterung → Basis-Use-Case

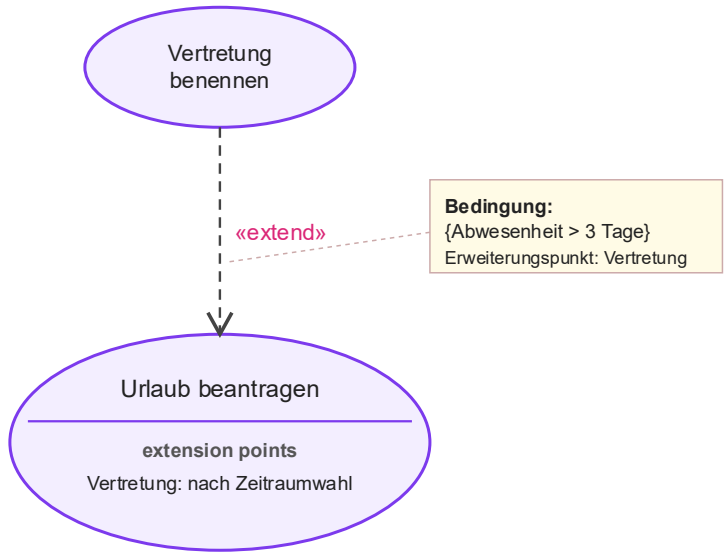
«extend»-Beziehung

- Symbol: gestrichelter Pfeil
- Richtung: vom erweiternden zum Basis-Use-Case

«extend» wird optional bzw. mit Bedingung ausgeführt.

- Extension Point: Stelle und Bedingung der Erweiterung (als Notiz oder im Use Case)

«extend» – Erweiterungspunkt im Use Case



Erweiterungspunkt steht im Basis-Use-Case, die Notiz verweist darauf

Stereotypen

<<include>> und <<extend>> nennt man Stereotypen.

Um UML erweiterbar zu machen, gibt es die s. g. Stereotypen – das sind spezielle Markierungen, die einem Element eine zusätzliche Bedeutung geben.

Diese Stereotypen werden immer mit doppelten spitzen Klammern << >> geschrieben, damit man sie sofort von normalen Bezeichnungen (z. B. Klassennamen oder Beziehungstypen) unterscheiden kann.

Merksatz

include = muss

extend = kann

Häufige Fehler & Best Practices

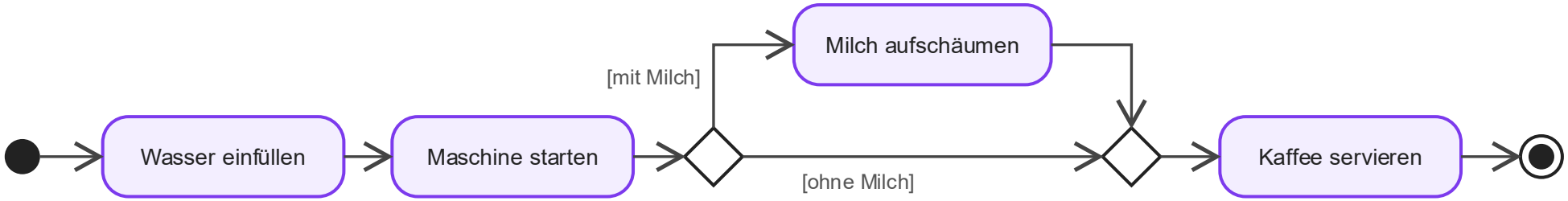
- Keine Abläufe modellieren: Use Cases haben keine Reihenfolge
- «include»/«extend» sparsam einsetzen (nicht für jede Kleinigkeit)
- Akteure als Rollen benennen, nicht als Personen
- Übersichtlich bleiben: max. ca. 10–15 Use Cases pro Diagramm
- Einheitliche Benennung von Use Cases: „Substantiv + Verb“

Verhaltensdiagramme

Aktivitätsdiagramm

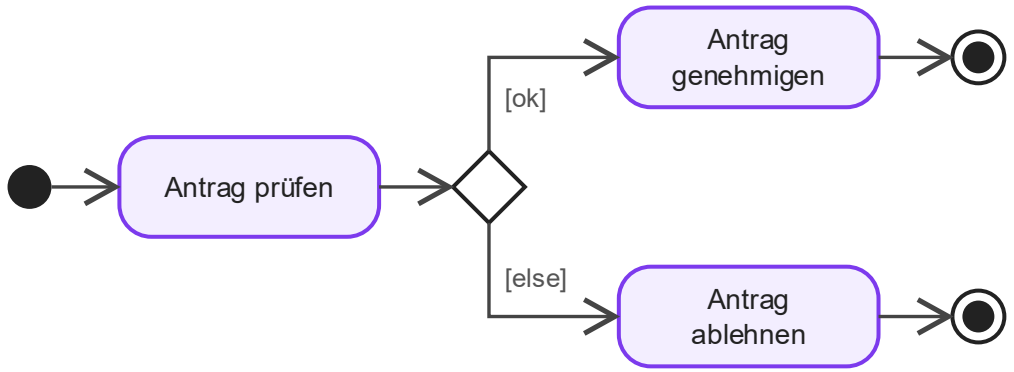
Einleitung

- Zeigt, wie etwas abläuft: Schritte und ihre Reihenfolge
- Typische Verwendung: Arbeitsabläufe, Systemabläufe und Algorithmen darstellen



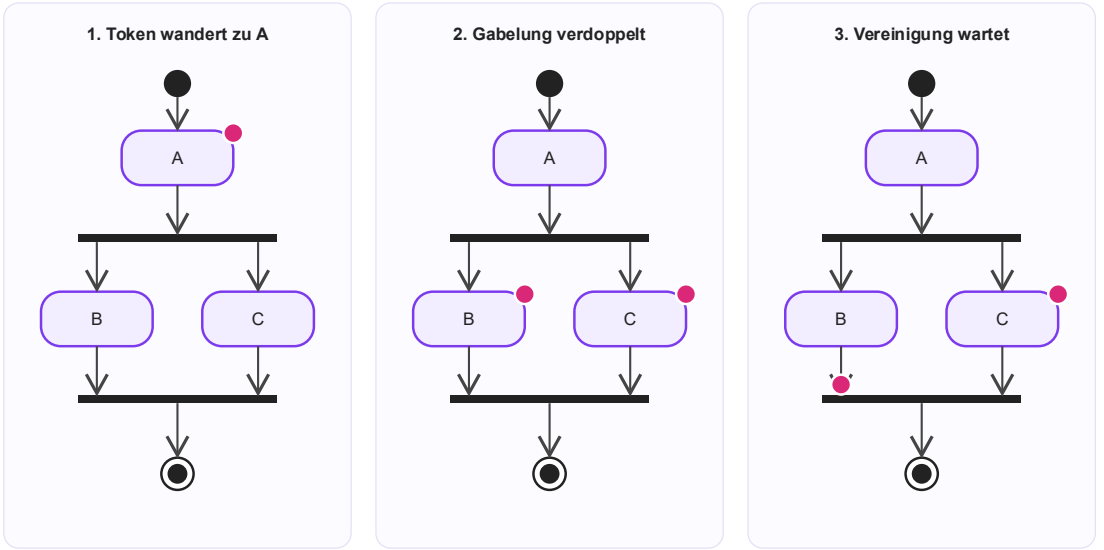
Einleitung

- Das Aktivitätsdiagramm stellt Abläufe bestehend aus Schritten, Entscheidungen, Parallelität und Verantwortlichkeiten dar.
- Typischer Einsatz: Anforderungen klären, Abläufe dokumentieren, Grundlage für Umsetzung und Test.



Das Token-Prinzip

- Ein Ablauf funktioniert wie ein Spielstein (Token), der durch das Diagramm wandert.
- Eine Aktion startet, wenn an allen eingehenden Kanten ein Token anliegt.
- Gabelung vervielfältigt Tokens.
- Vereinigung wartet, bis alle Tokens angekommen sind.



Die Grundelemente im Überblick



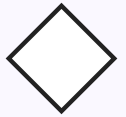
Startknoten
Beginn des Ablaufs



Aktion
ein Arbeitsschritt



Kontrollfluss
Reihenfolge



Entscheidung
entweder/oder



Gabelung/Vereinigung
sowohl/als auch



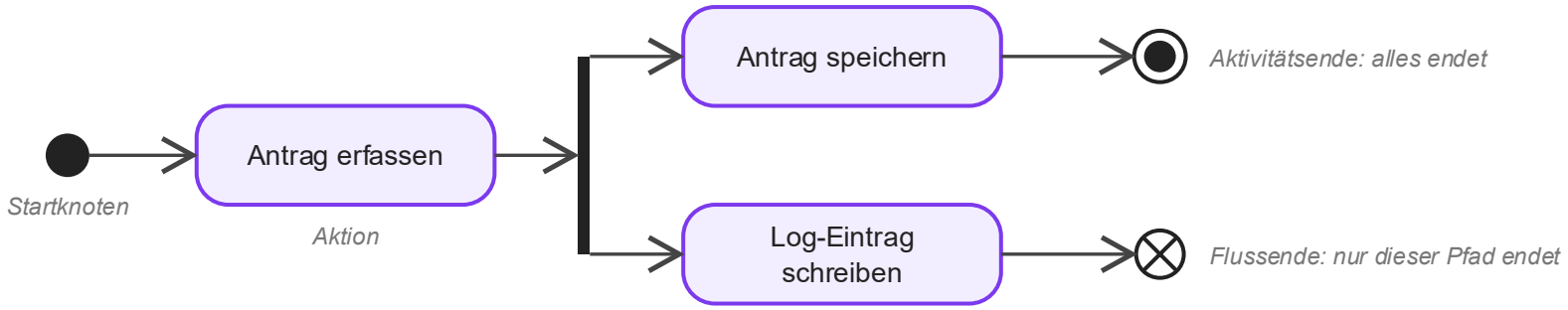
Endknoten
Aktivitäts-/Flussende



Partition
Verantwortung

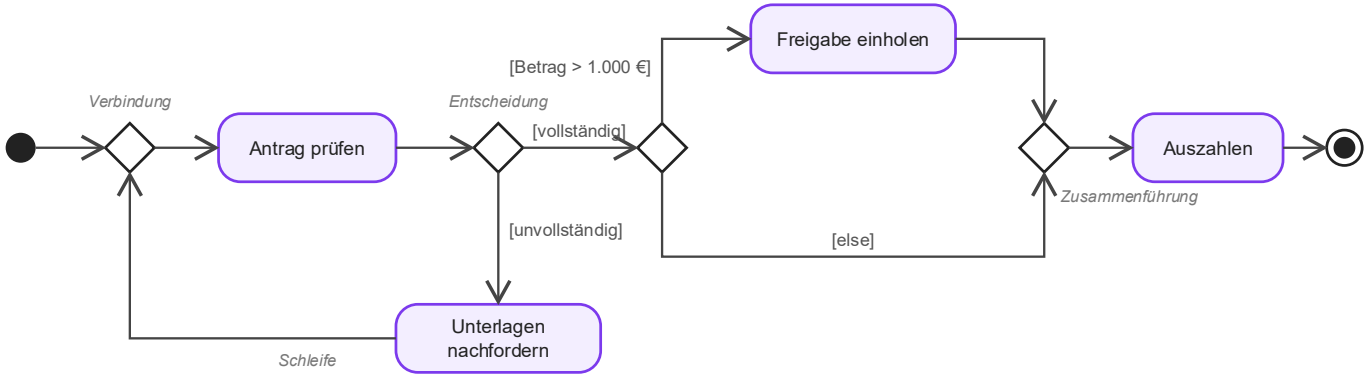
Start, Ende und Aktionen

- **Startknoten:** gefüllter schwarzer Kreis
- **Aktivitätsende:** Kreis mit gefülltem Punkt → beendet den gesamten Ablauf
- **Flussende:** Kreis mit X → beendet nur diesen einen Pfad
- **Aktion:** Rechteck mit abgerundeten Ecken, Verb + Objekt („Antrag prüfen“)
- **Kontrollfluss:** Pfeil zwischen den Knoten



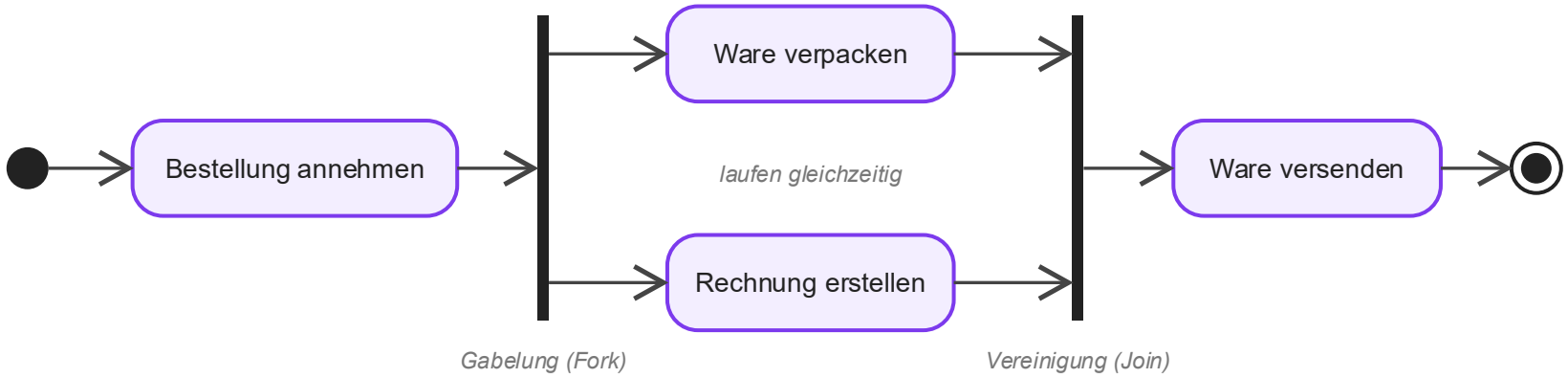
Entscheidung & Zusammenführung

- Entscheidungsknoten: Raute, ein Eingang, mehrere Ausgänge
- Guards in eckigen Klammern an den Ausgängen: [genehmigt], [abgelehnt]
- Guards schließen sich gegenseitig aus und decken alle Fälle ab
- Verbindungsknoten: Raute, mehrere Eingänge, ein Ausgang
- Schleifen: Rückweg zu einer früheren Aktion



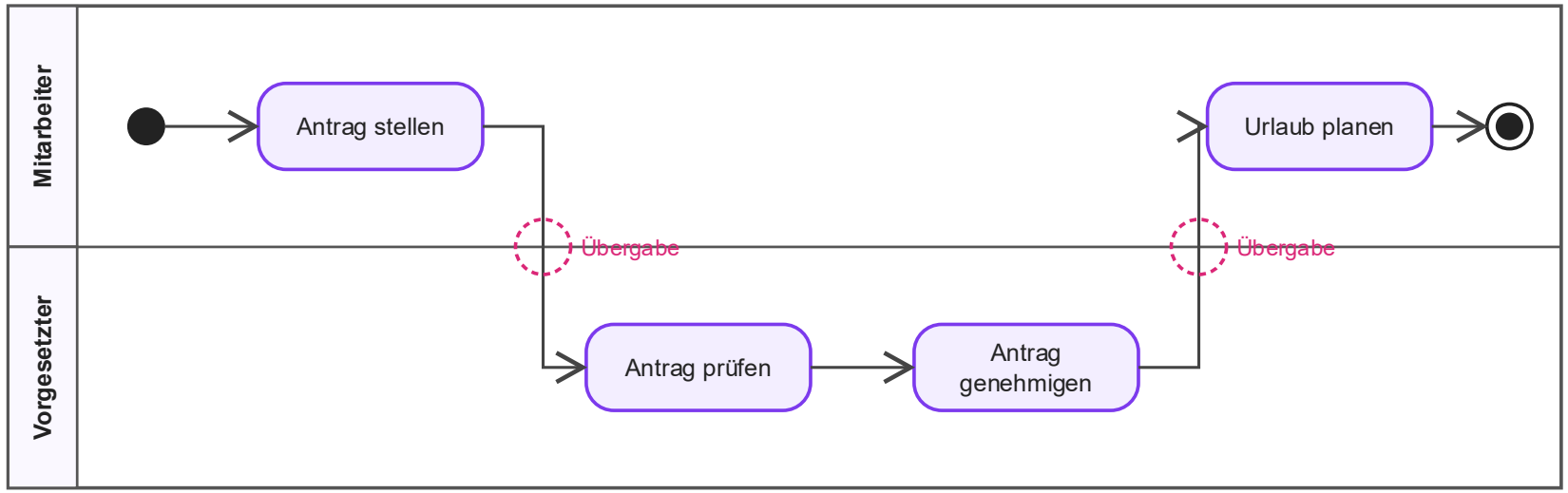
Gabelung & Vereinigung

- **Gabelung (Fork):**
  schwarzer Balken, ein Eingang, mehrere Ausgänge → alle Pfade laufen gleichzeitig
- **Vereinigung (Join):**
  schwarzer Balken, mehrere Eingänge, ein Ausgang → wartet, bis alle fertig sind
- **„Gleichzeitig“** heißt: unabhängig voneinander, Reihenfolge egal



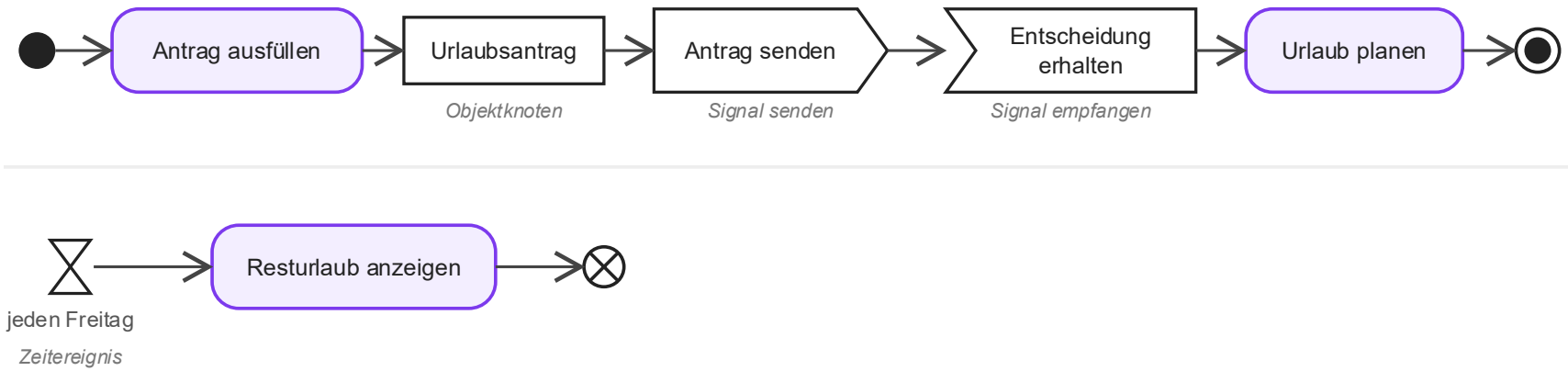
Partitionen (Swimlanes)

- Teilen das Diagramm in Verantwortungsbereiche (Rollen, Abteilungen, Systeme)
- Jede Aktion liegt in der Partition dessen, der sie ausführt
- Kontrollflüsse dürfen Partitionen überqueren – dort findet eine Übergabe statt

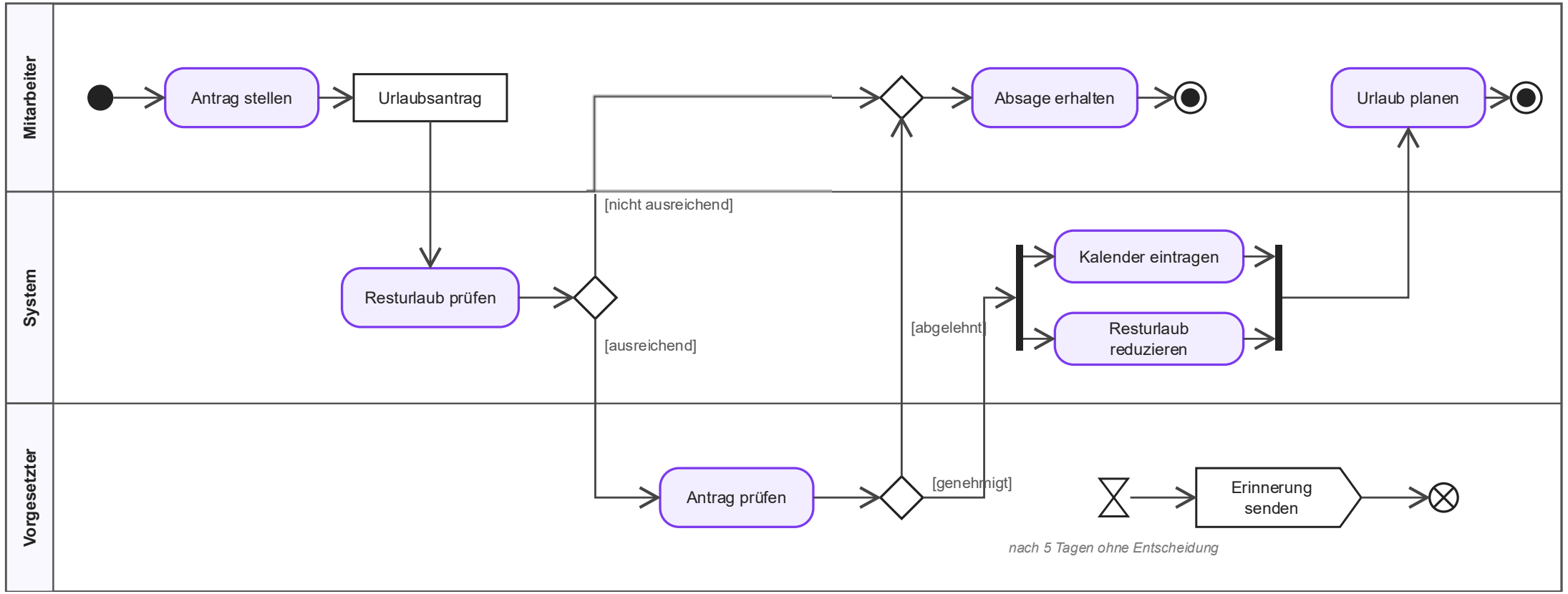


Objekte, Signale & Zeit

- **Objektknoten:** eckiges Rechteck (z. B. „Urlaubsantrag“) – Daten oder Dokumente im Fluss
- **Signal senden:** Fünfeck mit Spitze nach rechts („Bestätigung senden“)
- **Signal empfangen:** Rechteck mit Einkerbung links („Zahlung eingegangen“)
- **Zeitereignis:** Sanduhr („jeden Montag“, „nach 5 Tagen“)



Komplettbeispiel: Urlaubsantrag

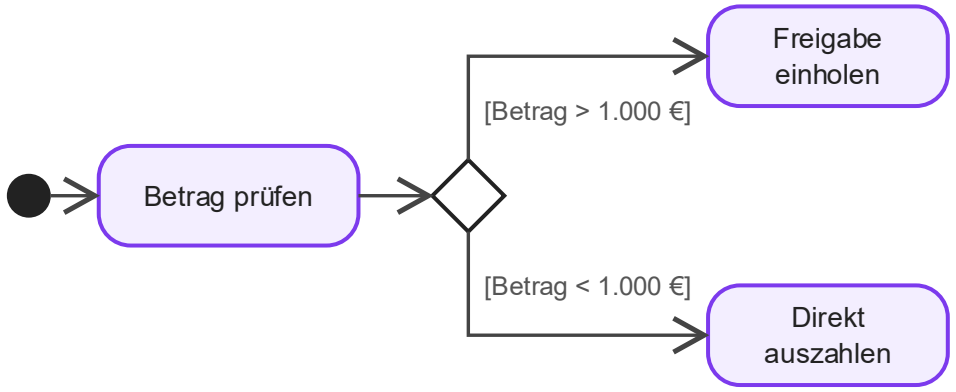


Häufige Fehler & Best Practices

- **Raute und Balken** nicht verwechseln (entweder/oder vs. sowohl/als auch)
- **Guards** vollständig und überschneidungsfrei formulieren
- **Jede Gabelung** braucht eine passende Vereinigung
- **Aktivitätsende** nur, wo wirklich alles endet (sonst Flussende)
- **Einheitliche Benennung:** "Verb + Objekt", nicht zu kleinteilig

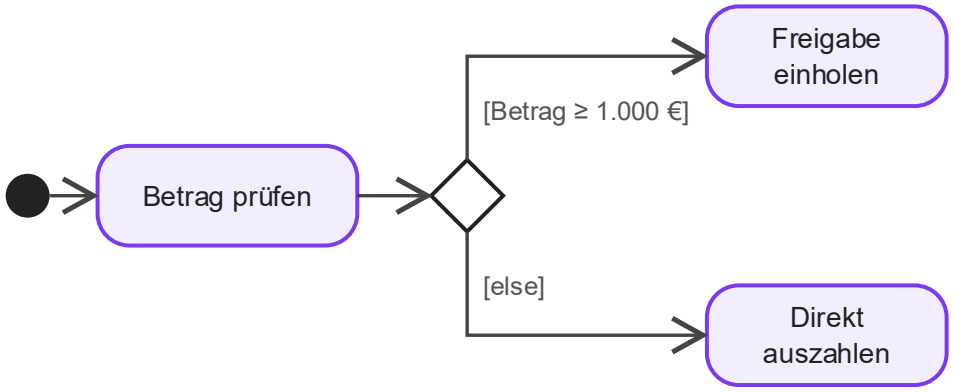
Guards & Schleifen richtig formulieren

Falsch



Genau 1.000 € ist nicht abgedeckt!

Richtig



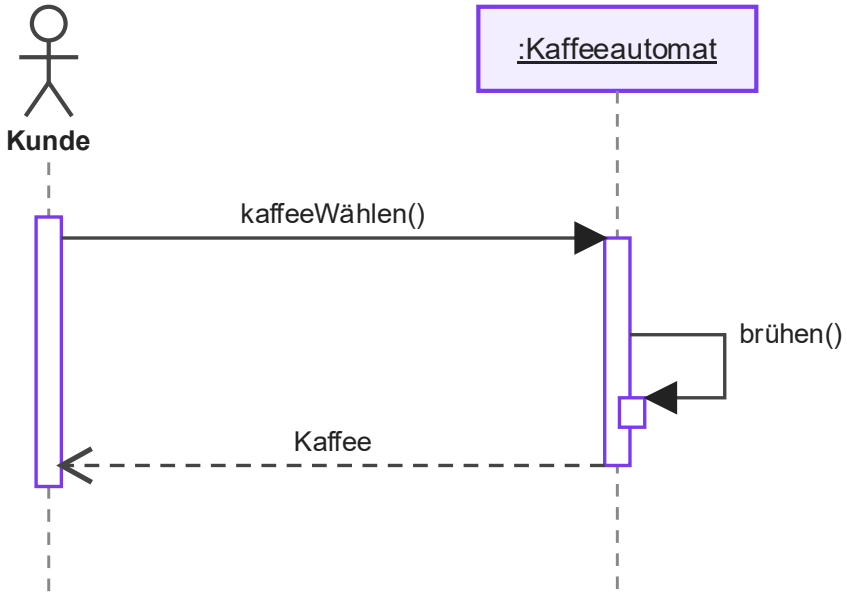
Jeder Fall ist abgedeckt

Verhaltensdiagramme

Sequenzdiagramm

Einleitung

- Zeigt den Nachrichtenaustausch zwischen Beteiligten
- Die Zeit läuft von oben nach unten, daher ist die Interaktions-Reihenfolge ist gut ablesbar



Einleitung

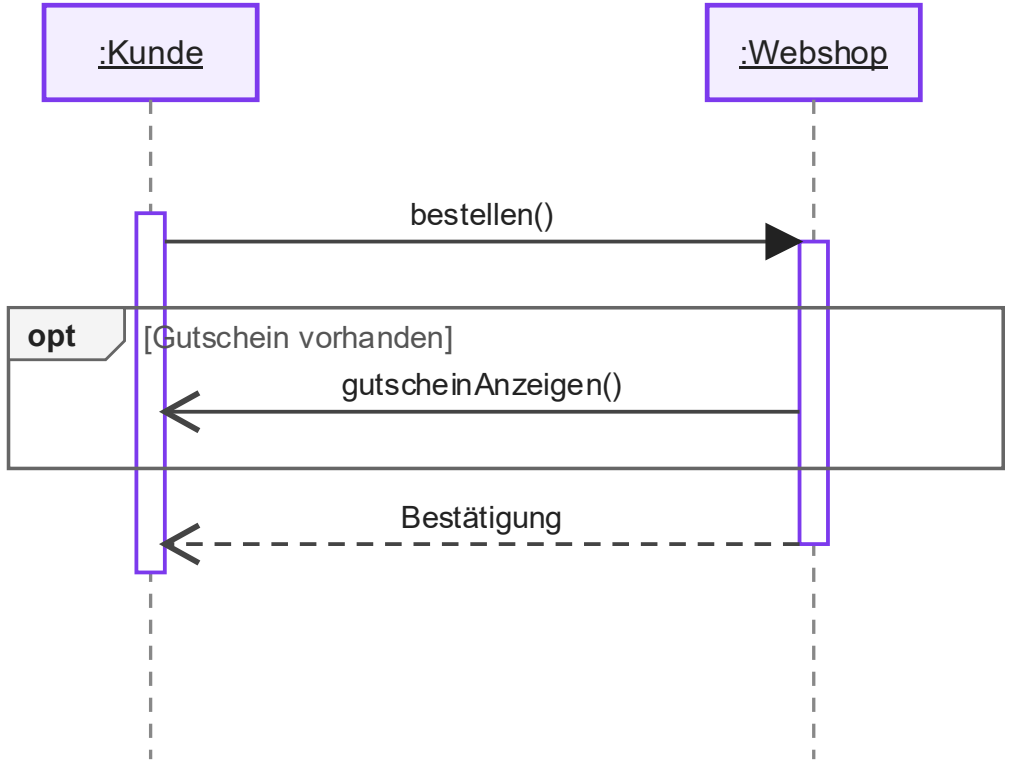
Das Sequenzdiagramm hilft uns bei:

- Zusammenspiel verstehen: welche Objekte oder Systeme wie kommunizieren
- Schnittstellen klären: welche Nachrichten mit welchen Parametern nötig sind
- Fehler finden: fehlende Antworten, falsche Reihenfolge, unnötige Aufrufe

Es ist damit die Grundlage für Umsetzung und Test.

Grundelemente

- **Lebenslinie:** ein Beteiligter mit Kopf und gestrichelter Linie
- **Nachricht:** Pfeil von Sender zu Empfänger
- **Antwort:** gestrichelter Pfeil zurück
- **Aktivierungsbalken:** zeigt, wann ein Beteiligter aktiv ist
- **Kombiniertes Fragment:** Rahmen für Bedingungen, Wiederholungen, Parallelität



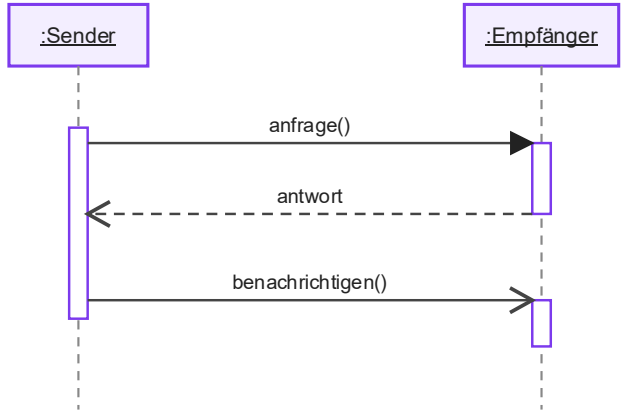
Lebenslinien

- **Kopf:** Rechteck mit unterstrichenem Namen
- **Akteur:** Strichmännchen für Personen außerhalb des Systems
- **Gestrichelte Linie** nach unten = Existenz über die Zeit
- **Anordnung:** Initiator links, weitere Beteiligte in Aufrufreihenfolge nach rechts



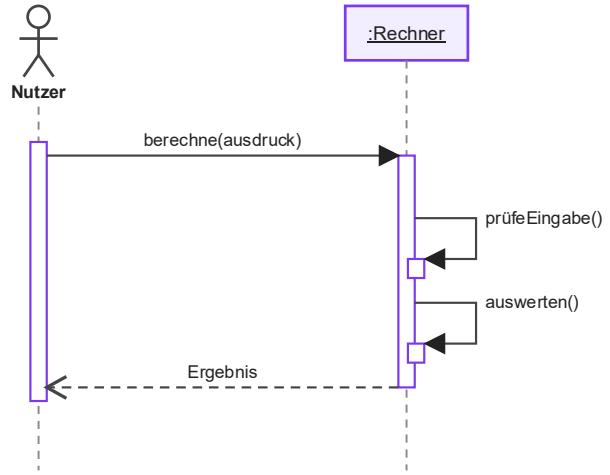
Nachrichtenarten

- Synchron: durchgezogene Linie, gefüllte Spitze → der Sender wartet auf die Antwort
- Asynchron: durchgezogene Linie, offene Spitze → der Sender arbeitet sofort weiter
- Antwort: gestrichelte Linie, offene Spitze
- Beschriftung: Name(Parameter), z. B. bestellen(artikel)



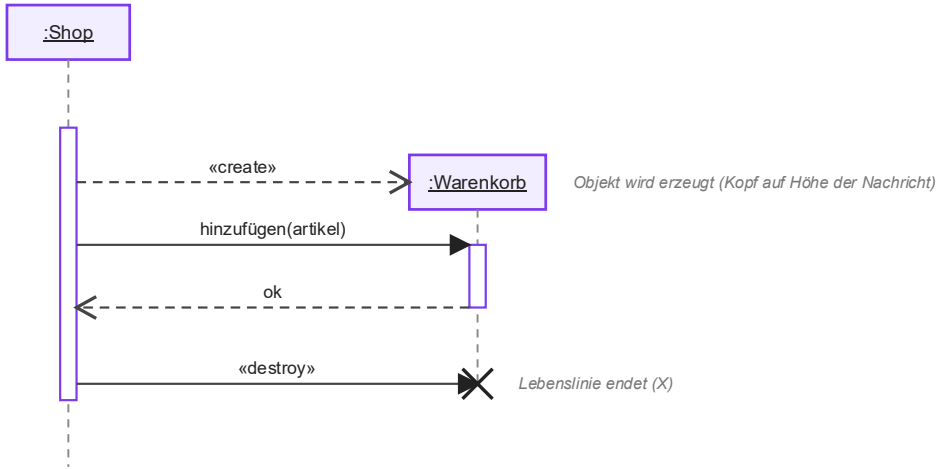
Aktivierung & Selbstaufruf

- Aktivierungsbalken: schmales Rechteck auf der Lebenslinie → der Beteiligte arbeitet gerade
- Beginnt mit der eingehenden Nachricht, endet mit der Antwort
- Selbstaufruf: Pfeil von der Lebenslinie zurück zu sich selbst
- Dabei entsteht ein verschachtelter Balken



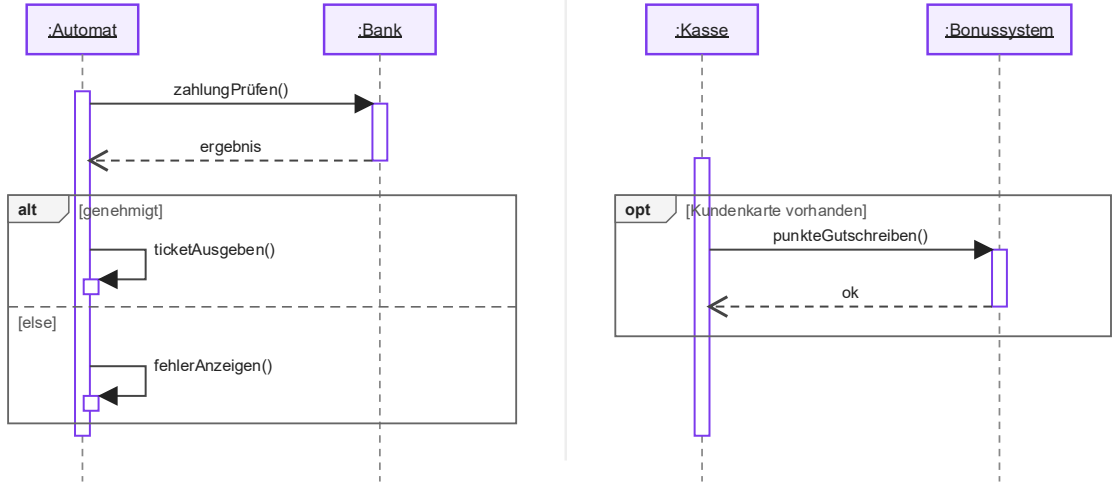
Erzeugen & Zerstören

- Erzeugen: gestrichelter Pfeil mit «create» direkt auf den Kopf des neuen Objekts
- Der Kopf steht auf Höhe der Nachricht, nicht oben
- Zerstören: großes X am Ende der Lebenslinie
- Typisch für kurzlebige Objekte, z. B. einen Warenkorb



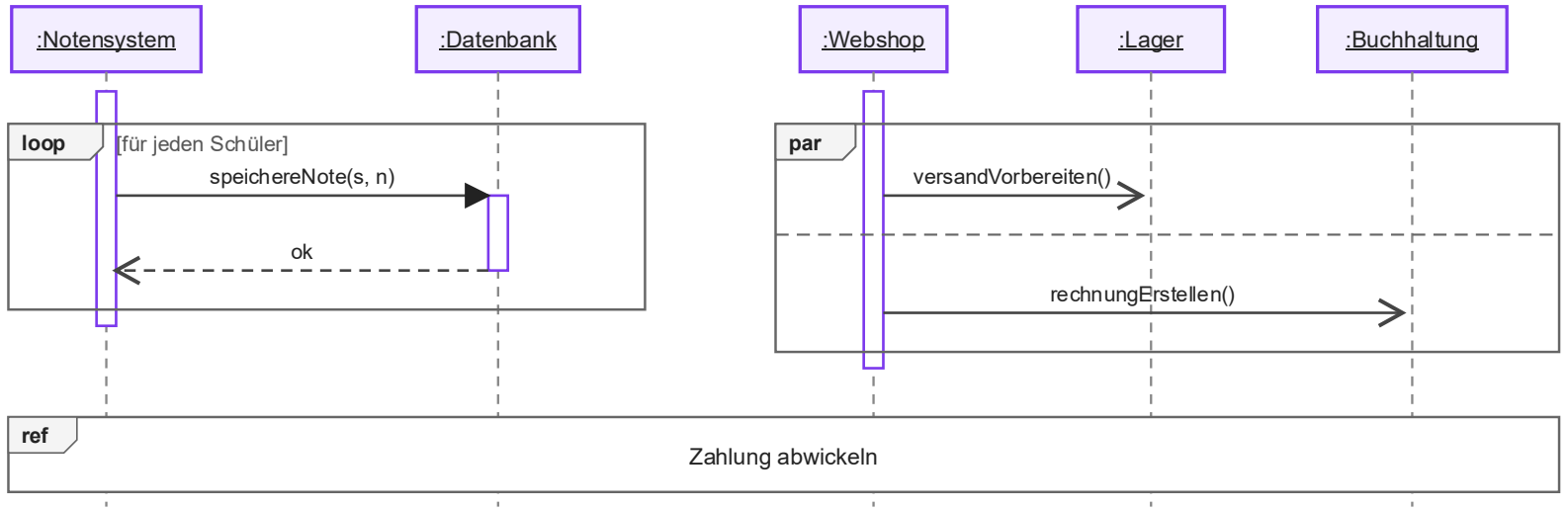
Kombinierte Fragmente: alt & opt

- Rahmen mit Operator oben links, Bedingungen (Guards) in eckigen Klammern
- alt: Alternativen – genau ein Abschnitt wird ausgeführt
- Abschnitte werden durch eine gestrichelte Linie getrennt, letzter oft [else]
- opt: optional – wird nur bei erfüllter Bedingung ausgeführt

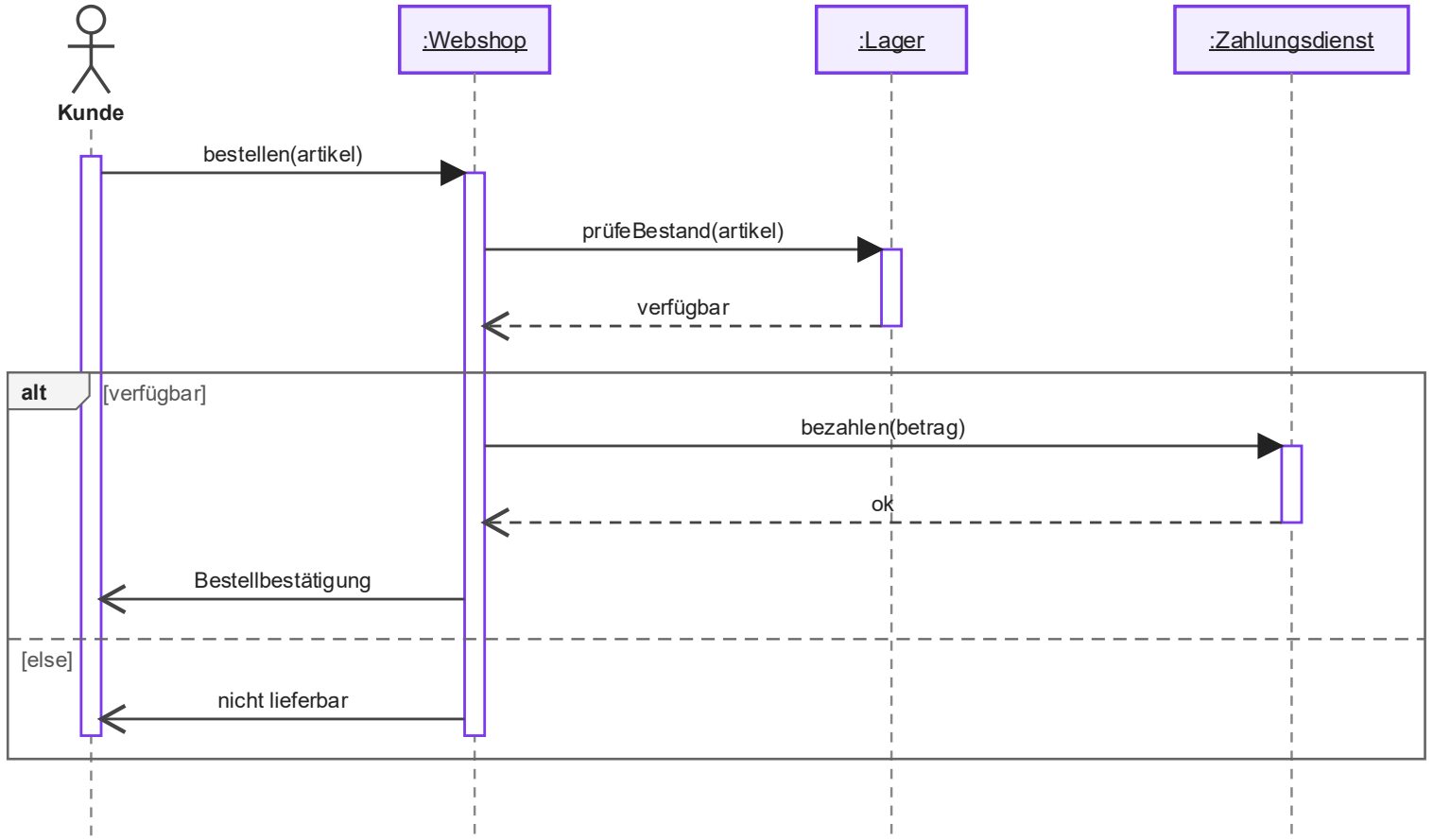


Kombinierte Fragmente: loop, par & ref

- loop: Wiederholung, z. B. [für jeden Schüler] oder loop(1,3)
- par: Abschnitte laufen parallel, getrennt durch gestrichelte Linien
- ref: Verweis auf ein anderes Sequenzdiagramm – hält große Diagramme übersichtlich



Beispiel: Online-Bestellung



Häufige Fehler & Best Practices

- Zeit läuft nur nach unten: keine Pfeile schräg nach oben
- Pfeilspitzen bewusst wählen: synchron gefüllt, asynchron offen
- Aktivierungsbalken passend zu Nachricht und Antwort setzen
- Nicht zu viel in ein Diagramm – Details mit ref auslagern
- Eindeutige Namen für Nachrichten und Lebenslinien

Tools

Das Erstellen von UML-Diagrammen ist nicht schwer, aber es erfordert Übung.

Also üben Sie 😜

- draw.io / diagrams.net: kostenlos, UML-Bibliothek integriert
- PlantUML: Sequenzdiagramme per Text
- Lucidchart, Visual Paradigm, Enterprise Architect: weitere Werkzeuge

12.10.2026

Ende